

Automated Water Refilling System — Project Overview

Enarxi Innovations Pvt. Ltd. | ESP32-based Embedded Firmware & Sensor Integration | Internship Project

1. Objective

Many rural areas lack borewell access and depend entirely on corporation-supplied water, which is available only for a limited window daily or on alternate days (30–60 minutes). Without a sump for storage, this water must be filled directly into overhead tanks — but unmonitored manual filling risks overflow (wasted supply) or missed fill windows (no water for the day). This project designs an ESP32-based system that automatically detects water availability and tank status, and fills the tank within the available supply window without manual monitoring.

2. System Architecture

Component	Details
MCU	ESP32 Dev Board
Sensors	Dry Run Sensor (GPIO 5, active-low) — water presence Float Sensor (GPIO 4, active-low) — tank full detection
Timekeeping	DS3231 RTC (I2C, 0x57/0x68) — required for Scheduled Mode
Display	16x2 LCD (I2C, 0x27) — mode, time, motor state, live events
Actuation	Relay-driven motor (GPIO 23, active-high)
User Input	3x pushbuttons — Schedule (16), Auto/Manual (17), Reset (18)
Indicators	5x single-color LEDs — Water, Tank Full, Motor, Manual, Scheduled
Comms	Serial (115200 baud) + Bluetooth ("Water Refilling System") — shared menu-driven config interface
Tools	Arduino IDE

3. Operation Logic (Control Flow Summary)

Auto Mode (default): Motor ON only if water present AND tank not full — direct threshold logic.

Manual Mode: Motor state directly toggled via button/BT command, sensors ignored.

Scheduled Mode: Motor runs at a fixed alarm time for a set duration regardless of sensors; post-run, continues if water present and tank not full, else retries (0–5 attempts, 1–60 min interval) until success or failure logged. Requires RTC — falls back to Auto Mode if unavailable.

Safety override: mid-run, motor stops immediately if tank becomes full or water disappears, independent of active mode.

4. My Contribution

- Firmware logic design for sensor-driven state transitions across three operation modes
- Sensor integration and validation — I2C (RTC, LCD) and GPIO digital sensing (water, float)
- Implementation of retry-with-backoff logic for scheduled-run failure recovery
- Menu-driven configuration system over Serial and Bluetooth (shared command parser)
- Utilized ESP32's dual-core architecture, running the Bluetooth task on a separate core for responsive input handling without blocking the main control loop

5. Key Challenges & Solutions

Challenge	Solution
False motor cycling on sensor noise	5s state-confirmation delay before logging/acting on state change
RTC dependency for Scheduled Mode	Graceful fallback to Auto Mode with explicit "RTC Not Found" flag
Dual input channels (Serial + BT)	Shared menu state machine keyed per-stream to avoid conflicts
LCD flicker on frequent status updates	Selective line redraw instead of full-screen clear, only on content change
Supply-window failure (no water at scheduled time)	Retry-with-backoff (configurable count/interval) so brief delays don't abort the fill cycle

6. Future Work

- Implementation of an analog sensor to determine non-binary tank status (fill level %) and water pressure, replacing binary float/dry-run detection
- Address single-point-of-failure risk in dry-run/float sensors: propose fault intimation (LCD/BT alert) on sensor failure. Add an alternative detection method (e.g., timer-based estimation or redundant sensor) so the system degrades safely instead of failing silently.
- Persist settings to non-volatile storage (currently reset on power loss)
- Mobile app front-end replacing raw Bluetooth terminal commands